De los structs a los objetos

Structs, métodos, templates y namespaces en C++

Fundamentos de Programación
Escuela de Ingeniería de Sistemas e Informática
Universidad Industrial de Santander

Agenda

1. **Structs**: agrupar datos que van juntos
2. **Métodos**: funciones que viven dentro del struct
3. **Templates**: una sola definición para muchos tipos
4. **Namespaces**: apellidos para los nombres
5. Ejemplo integrador, errores comunes y ejercicios

> Hoy **no** se verá herencia, polimorfismo ni encapsulamiento con `private`. Eso llegará más adelante.

1. Structs

Cuando varios datos describen una misma cosa

El problema: datos que van juntos

Se quiere guardar el nombre, el código y la nota de **un** estudiante:

```
string nombre = "Ana";
int codigo = 2251001;
double nota = 4.2;
```

¿Y para 40 estudiantes? Con lo visto hasta ahora habría que usar **tres arreglos paralelos**:

```
vector<string> nombres;
vector<int> codigos;
vector<double> notas;
// nombres[i], codigos[i] y notas[i] "deberían" ser del mismo estudiante...
```

La idea: un solo paquete

Sin struct: variables sueltas

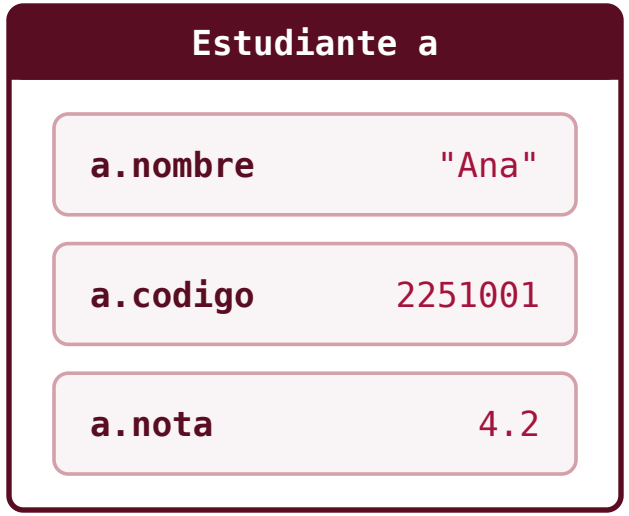
```
string nombre
"Ana"
```

```
int codigo
2251001
```

```
double nota
4.2
```

¿Cuál nota es de cuál estudiante?

Con struct: un solo paquete



Los datos viajan juntos y tienen nombre

Definir un struct

```
struct Estudiante {
    string nombre;
    int codigo;
    double nota;
};   // ← ¡el punto y coma es obligatorio!
```

- `struct Estudiante` crea un **tipo nuevo** llamado `Estudiante`
- `nombre`, `codigo` y `nota` son sus **campos** (o **atributos**)
- La definición va **fuera** de `main`, normalmente al inicio del archivo
- Por convención, el nombre del tipo empieza con **mayúscula**

Crear variables y usar el operador punto

```
Estudiante a;              // una variable de tipo Estudiante
a.nombre = "Ana";          // el punto entra a un campo
a.codigo = 2251001;
a.nota = 4.2;

Estudiante b{"Luis", 2251002, 3.4};   // inicialización con llaves

cout << b.nombre << " tiene " << b.nota << endl;
```

- `variable.campo` se usa exactamente como una variable normal
- Con llaves `{}` los valores se asignan **en el orden** en que se declararon los campos

Structs y funciones

```
// Referencia constante: no copia y no modifica
void imprimir(const Estudiante &e) {
    cout << e.codigo << "  " << e.nombre << "  " << e.nota << endl;
}

// Referencia: sí modifica el original
void bonificar(Estudiante &e, double puntos) {
    e.nota = e.nota + puntos;
    if (e.nota > 5.0) {
        e.nota = 5.0;
    }
}
```

Las reglas de paso de parámetros son **las mismas** que ya conocen.

Un vector de structs

```
vector<Estudiante> grupo;
grupo.push_back({"Ana", 2251001, 4.2});
grupo.push_back({"Luis", 2251002, 3.4});
grupo.push_back({"Marta", 2251003, 4.7});

bonificar(grupo[1], 0.3);

double suma = 0.0;
for (const Estudiante &e : grupo) {
    suma += e.nota;
}
cout << "Promedio: " << suma / grupo.size() << endl;   // 4.2
```

Ahora cada elemento del vector es **un estudiante completo**: ya no hay arreglos paralelos.

2. Métodos

Funciones que viven dentro del struct

Ustedes ya han usado métodos

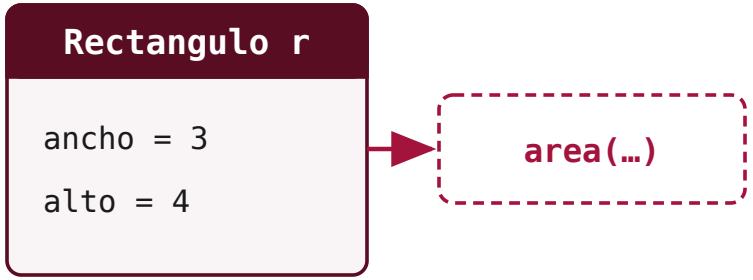
```
vector<int> v{4, 8, 15};
v.push_back(16);
cout << v.size() << endl;

string s = "hola";
cout << s.length() << endl;
```

- `push_back`, `size` y `length` **no** se llaman así: `size(v)`
- Se llaman **sobre** la variable: `v.size()`
- Son **funciones que pertenecen al dato**: eso es un **método**

Función libre vs. método

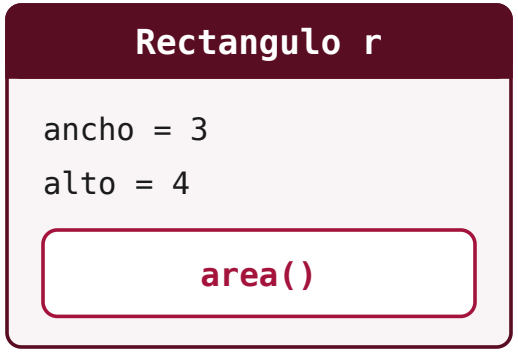
Función libre



`area(r)`

La función está afuera;
hay que entregarle el dato.

Método



`r.area()`

La función vive dentro del struct;
ya conoce ancho y alto.

El primer método

```
struct Rectangulo {
    double ancho;
    double alto;

    double area() {
        return ancho * alto;     // usa los campos directamente
    }
};

int main() {
    Rectangulo r{3.0, 4.0};
    cout << r.area() << endl;    // 12
}
```

- El método se escribe **dentro** de las llaves del struct
- Dentro del método, `ancho` y `alto` son los campos **del objeto que lo llamó**

Comparación lado a lado

Con función libre

```
double area(const Rectangulo &r) {
    return r.ancho * r.alto;
}
cout << area(r) << endl;
```

Con método

```
struct Rectangulo {
    double ancho;
    double alto;
    double area() { return ancho * alto; }
};
cout << r.area() << endl;
```

Métodos que modifican el objeto

```
struct Rectangulo {
    double ancho;
    double alto;

    double area() { return ancho * alto; }

    void escalar(double factor) {
        ancho = ancho * factor;
        alto = alto * factor;
    }
};

Rectangulo r{3.0, 4.0};
r.escalar(2.0);
cout << r.ancho << " x " << r.alto << endl;   // 6 x 8
cout << r.area() << endl;                      // 48
```

Métodos `const`: "solo miro, no toco"

```
struct Rectangulo {
    double ancho;
    double alto;

    double area() const {          // promete no modificar el objeto
        return ancho * alto;
    }

    void escalar(double factor) {  // modifica: no lleva const
        ancho = ancho * factor;
        alto = alto * factor;
    }
};
```

- Se pone `const` **después** de los paréntesis en los métodos que solo consultan
- Si un método `const` intenta modificar un campo, el compilador da **error**

Un método puede usar otro método

```
struct Estudiante {
    string nombre;
    vector<double> notas;

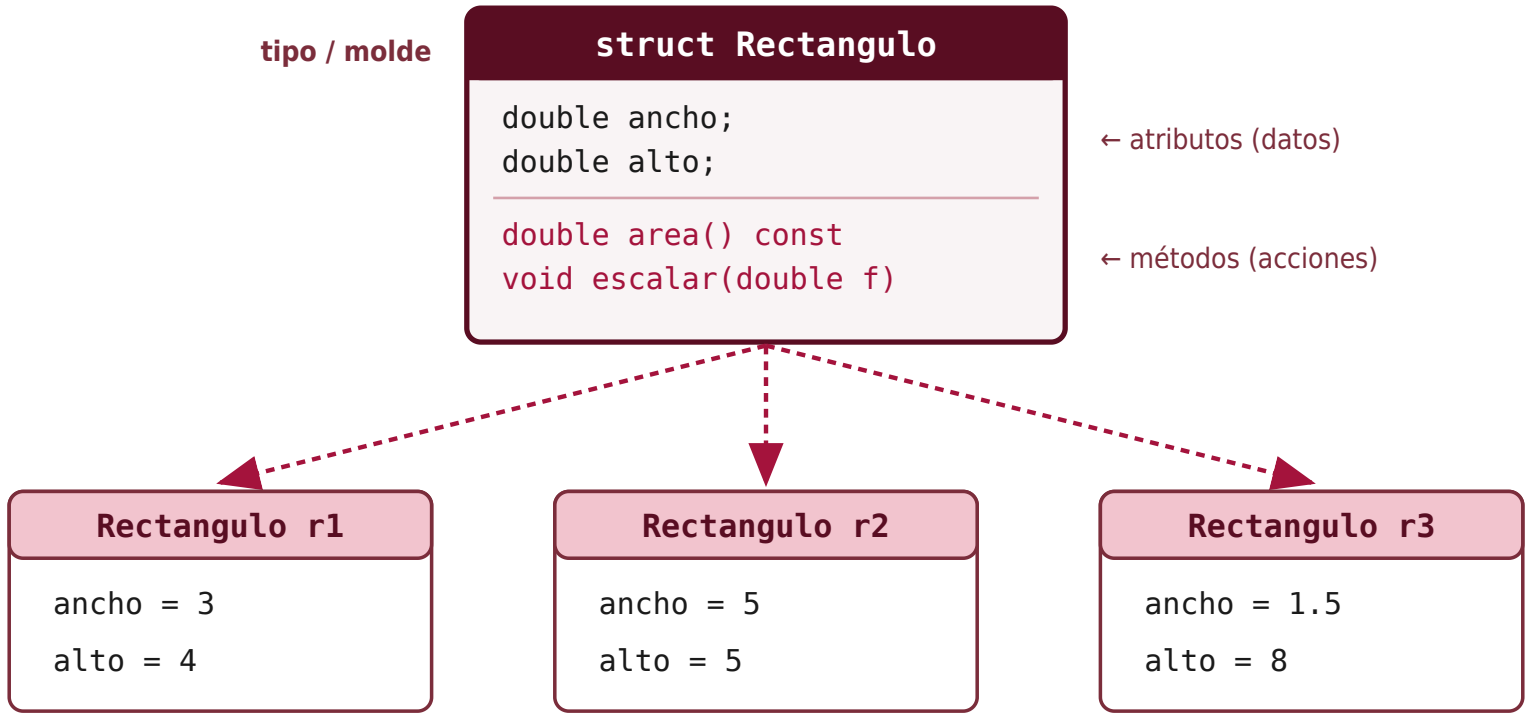
    void agregarNota(double n) { notas.push_back(n); }

    double promedio() const {
        if (notas.empty()) return 0.0;
        double suma = 0.0;
        for (double n : notas) suma += n;
        return suma / notas.size();
    }

    bool aprobo() const { return promedio() >= 3.0; }
};
```

```
Estudiante e{"Ana", {}};
e.agregarNota(3.5);  e.agregarNota(4.0);  e.agregarNota(2.8);
cout << e.promedio() << " " << e.aprobo() << endl;   // 3.43333 1
```

Vocabulario: tipo y objetos



objetos (instancias): cada uno con sus propios valores, todos con los mismos métodos

En resumen: un objeto es...

Término	Qué es	Ejemplo
Tipo (struct / clase)	El molde	`Rectangulo`
Objeto (instancia)	Una variable de ese tipo	`r1`, `r2`
Atributo (campo)	Un dato dentro del objeto	`r1.ancho`
Método	Una función dentro del objeto	`r1.area()`

> **Objeto = datos + las funciones que trabajan sobre esos datos**

3. Templates

Una sola definición para muchos tipos

El problema: el mismo código, tipos distintos

```
int mayor(int a, int b) {
    if (a > b) return a;
    return b;
}

double mayor(double a, double b) {
    if (a > b) return a;
    return b;
}

string mayor(string a, string b) {
    if (a > b) return a;
    return b;
}
```

Tres funciones **idénticas**; solo cambia el tipo. ¿Y si mañana se necesita para `char`?

La solución: un template de función

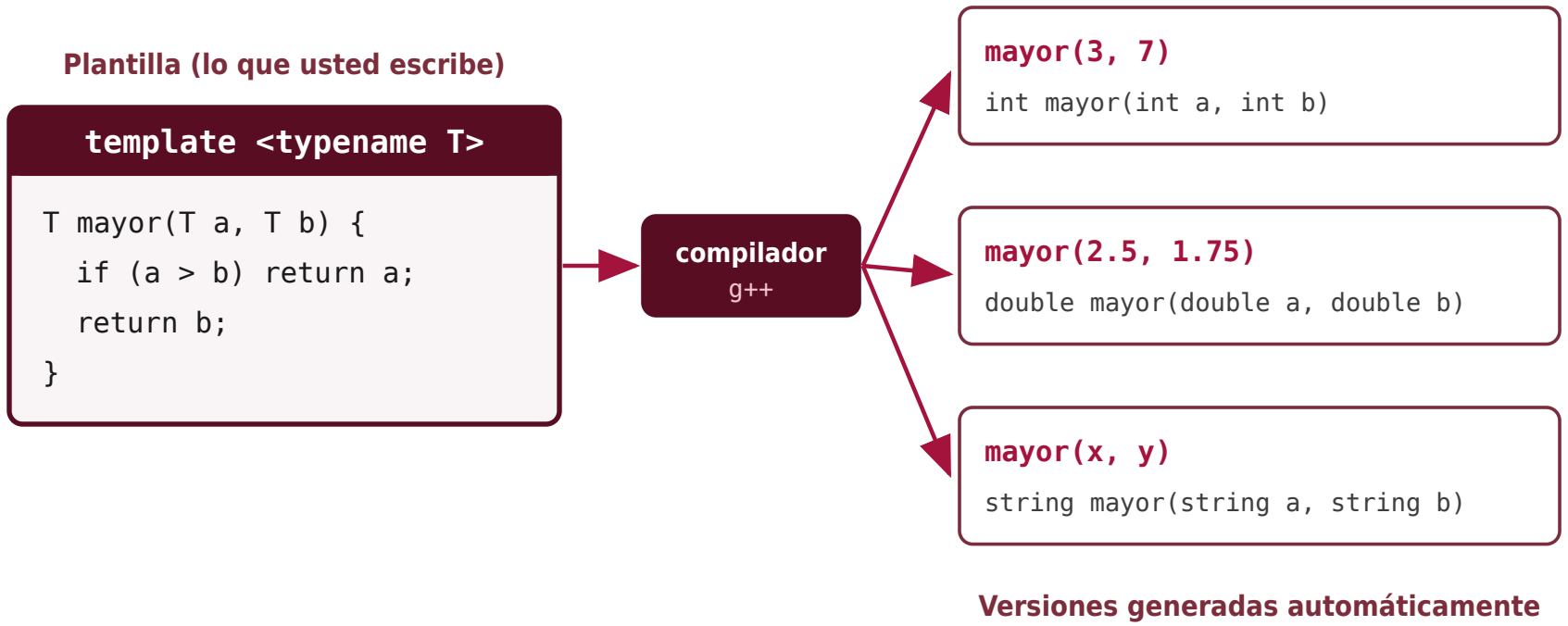
```
template <typename T>
T mayor(T a, T b) {
    if (a > b) return a;
    return b;
}

int main() {
    cout << mayor(3, 7) << endl;         // T = int     → 7
    cout << mayor(2.5, 1.75) << endl;    // T = double  → 2.5

    string x = "perro", y = "gato";
    cout << mayor(x, y) << endl;         // T = string  → perro
}
```

- `template <typename T>` dice: "`T` es un tipo que se decide después"
- Dentro de la función, `T` se usa como cualquier otro tipo

¿Qué hace el compilador?



Ustedes ya usan templates: `vector<T>`

```
vector<int> enteros{1, 2, 3, 4};
vector<double> reales{0.5, 0.25, 0.25};
vector<string> palabras{"hola", " ", "mundo"};
```

`vector` **no** es un tipo: es una **plantilla**. Los ángulos `< >` le dicen qué tipo usar.

```
template <typename T>
T sumar(const vector<T> &v) {
    T total{};                  // el "cero" del tipo T
    for (const T &x : v) {
        total += x;
    }
    return total;
}
// sumar(enteros) → 10   sumar(reales) → 1   sumar(palabras) → "hola mundo"
```

Templates de struct

```
template <typename T>
struct Par {
    T primero;
    T segundo;

    T elMayor() const {
        return mayor(primero, segundo);   // usa el template anterior
    }
};

int main() {
    Par<int> p{3, 8};
    Par<string> n{"Ana", "Luis"};
    cout << p.elMayor() << " " << n.elMayor() << endl;   // 8 Luis
}
```

Con structs el tipo se escribe **explícitamente** entre ángulos: `Par<int>`, como en `vector<int>`.

Cuidados con los templates

```
mayor(3, 2.5);   // ERROR: ¿T es int o double?
```

```
error: no matching function for call to 'mayor(int, double)'
```

- Los dos argumentos deben ser del **mismo tipo** `T`
- Solución: indicar el tipo a mano → `mayor<double>(3, 2.5)`

- `T` debe **soportar** las operaciones que se usan dentro: `mayor` usa `>`, así que no sirve con un `Rectangulo` (no se sabe cuál es "mayor")
- El error aparece **al compilar**, en la línea donde se usa el template

4. Namespaces

Apellidos para los nombres

El problema: nombres que chocan

En un programa grande, dos personas escriben una función `area`:

```
// Escrita por la persona A: área de un rectángulo
double area(double base, double altura) {
    return base * altura;
}

// Escrita por la persona B: área de un triángulo
double area(double base, double altura) {
    return base * altura / 2.0;
}
```

```
error: redefinition of 'double area(double, double)'
```

Mismo nombre, mismos parámetros: el compilador no puede distinguirlas.

La solución: `namespace`

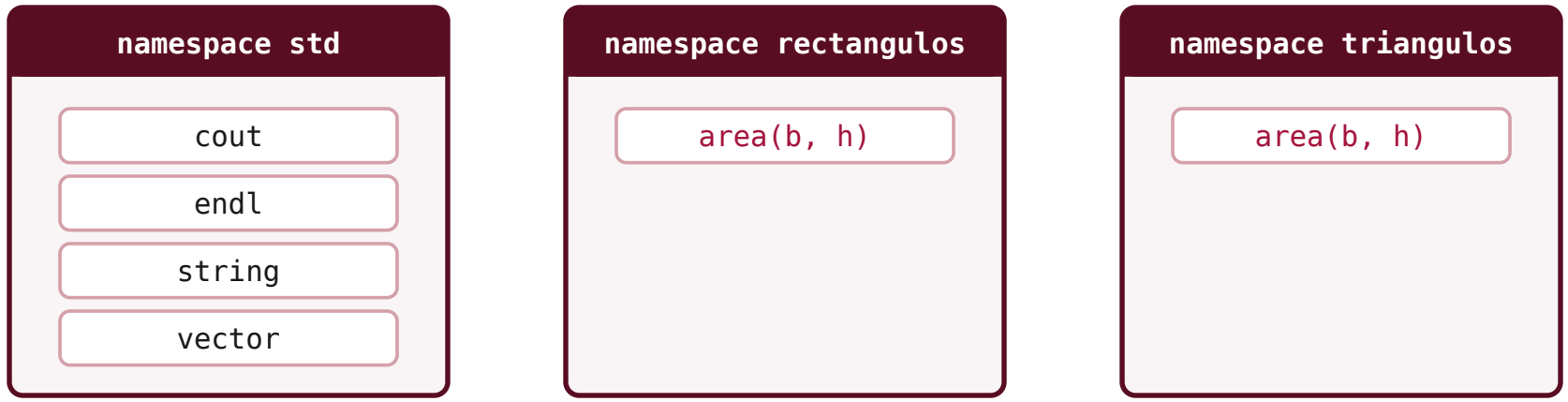
```
namespace rectangulos {
    double area(double base, double altura) {
        return base * altura;
    }
}

namespace triangulos {
    double area(double base, double altura) {
        return base * altura / 2.0;
    }
}

int main() {
    cout << rectangulos::area(3.0, 4.0) << endl;   // 12
    cout << triangulos::area(3.0, 4.0) << endl;    // 6
}
```

El operador `::` significa "**el que está dentro de**".

Cajas con nombre



Mismo nombre, distinta caja: el operador :: dice de cuál caja se toma

Entonces, ¿qué es `using namespace std;`?

```
using namespace std;       // "abra toda la caja std"
cout << "hola" << endl;   // ya no hace falta std::
```

Es cómodo, pero **trae todos los nombres** de `std` al programa y reabre la posibilidad de choques.

Alternativas más cuidadosas:

```
std::cout << "hola" << std::endl;   // 1. escribir el apellido siempre

using std::cout;                   // 2. traer solo lo que se usa
using std::endl;
cout << "hola" << endl;
```

> En ejercicios cortos `using namespace std;` es aceptable. En proyectos grandes y en archivos `.h`, **evítelo**.

5. Todo junto

Ejemplo integrador, errores comunes y ejercicios

Ejemplo integrador (1/2): la definición

```
namespace geo {                          // namespace: la caja

    template <typename T>                // template: coordenadas de cualquier tipo
    struct Punto {                       // struct: los datos
        T x;
        T y;

        void mover(T dx, T dy) {         // método que modifica
            x = x + dx;
            y = y + dy;
        }

        double distancia(const Punto<T> &otro) const {   // método que consulta
            double ddx = otro.x - x;
            double ddy = otro.y - y;
            return std::sqrt(ddx * ddx + ddy * ddy);
        }
    };

}
```

Ejemplo integrador (2/2): el uso

```
int main() {
    geo::Punto<int> casilla{2, 3};              // namespace :: struct < tipo >
    casilla.mover(1, 1);
    std::cout << casilla.x << ", " << casilla.y << "\n";   // 3, 4

    geo::Punto<double> a{0.0, 0.0};
    geo::Punto<double> b{3.0, 4.0};
    std::cout << a.distancia(b) << "\n";         // 5
}
```

- `geo::` → se busca dentro del namespace `geo`
- `Punto<int>` y `Punto<double>` → dos tipos generados por la misma plantilla
- `casilla.mover(1, 1)` y `a.distancia(b)` → métodos llamados con el punto

Errores comunes

Error	Mensaje típico de g++
Olvidar el `;` al final del struct	`expected ';' after struct definition`
Olvidar los paréntesis: `r.area`	`invalid use of non-static member function`
Modificar un campo en un método `const`	`assignment of member ... in read-only object`
Mezclar tipos en un template: `mayor(3, 2.5)`	`no matching function for call to 'mayor(int, double)'`
Olvidar el namespace: `area(3, 4)`	`'area' was not declared in this scope`

Ejercicios propuestos

1. Defina `struct Fecha` con día, mes y año, y un método `bool esBisiesto() const`.
2. Defina `struct CuentaBancaria` con titular y saldo, y métodos `depositar(double)`, `retirar(double)` (que no deje el saldo negativo) y `mostrar() const`.
3. Agregue a `Estudiante` un método `double mejorNota() const` y úselo sobre un `vector<Estudiante>` para encontrar al mejor estudiante del grupo.
4. Escriba un template `T menorDeVector(const vector<T> &v)` y pruébelo con `int`, `double` y `string`.
5. Ponga los ejercicios 1 y 2 dentro de un `namespace banco` y otro `namespace calendario`, y úselos desde `main` con `::`.

Lo que viene después

Lo visto hoy es la base. La programación orientada a objetos agrega, entre otras cosas:

- `class` y `private`: ocultar los atributos para que solo los métodos los cambien
- **Constructores**: métodos especiales que inicializan el objeto al crearlo
- **Herencia**: crear tipos nuevos a partir de otros
- **Polimorfismo**: tratar de forma uniforme objetos de tipos distintos

> Todo eso parte de la misma idea de hoy: **datos + funciones que trabajan sobre esos datos**.

¿Preguntas?

Structs · Métodos · Templates · Namespaces

Los ejemplos completos están en la carpeta `ejemplos/`

Compilar con: `g++ -std=c++17 -Wall archivo.cpp -o programa`